

Tabla Taala — 01 Model Training

Train a small AudioCNN on the Tabla Taala dataset using mel-spectrograms, SpecAugment, label smoothing, and class weights.

```
In [76]: # 1) Imports & Config
import os, json, math, random
from pathlib import Path

# Environment toggles (set before importing TensorFlow)
os.environ.setdefault('KERAS_BACKEND', 'tensorflow')
os.environ.setdefault('TF_DETERMINISTIC_OPS', '1')
os.environ.setdefault('TF_CUDNN_DETERMINISTIC', '1')
os.environ.setdefault('TRAIN_ON_GPU', '1')
os.environ.setdefault('TAB_FORCE_CPU', '0')
os.environ.setdefault('TAB_USE_MIXED_PRECISION', '1')
os.environ.setdefault('TAB_ENABLE_TF32', '1')
# Checkpoint/training flow toggles
os.environ.setdefault('TAB_FORCE_RETRAIN', '0')
os.environ.setdefault('TAB_RESUME_FROM_CKPT', '0')
os.environ.setdefault('SKIP_TRAIN_IF_CKPT', '0')
# Data/cache/aug toggles
os.environ.setdefault('TAB_CACHE_TRAIN', '0') # default off to keep per-epoch
os.environ.setdefault('TAB_SPEC_FREQ_MASK', '4') # soften augmentation for
os.environ.setdefault('TAB_SPEC_TIME_MASK', '8')
os.environ.setdefault('TAB_SPEC_NUM_MASKS', '1')
# Init from GTZAN checkpoint (transfer feature weights; final layer skipped)
os.environ.setdefault('TAB_INIT_FROM_GTZAN', '1')

if os.environ.get('TAB_FORCE_CPU', '0') == '1' or os.environ.get('TRAIN_ON_GPU', '0') == '0':
    os.environ['CUDA_VISIBLE_DEVICES'] = '-1'

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import librosa
import tensorflow as tf
import keras
from keras import layers, models
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.utils.class_weight import compute_class_weight

# Seeds and reproducibility
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
random.seed(RANDOM_STATE)
try:
    tf.random.set_seed(RANDOM_STATE)
except Exception:
    pass
```

```

# Optional: GPU memory growth and mixed precision
try:
    gpus = tf.config.list_physical_devices('GPU') if os.environ.get('TAB_FORCE_GPU') == '1':
    for g in gpus:
        tf.config.experimental.set_memory_growth(g, True)
    if gpus:
        print(f'GPUs found: {len(gpus)} - enabled memory growth')
        if os.environ.get('TAB_USE_MIXED_PRECISION', '1') == '1':
            try:
                from keras import mixed_precision
                mixed_precision.set_global_policy('mixed_float16')
                print('Mixed precision policy: mixed_float16')
            except Exception as e:
                print('Mixed precision unavailable:', e)
        if os.environ.get('TAB_ENABLE_TF32', '1') == '1':
            try:
                tf.config.experimental.enable_tensor_float_32_execution(True)
                print('TF32 enabled')
            except Exception as e:
                print('TF32 enable failed:', e)
    else:
        print('No GPUs visible or CPU forced; running on CPU.')
except Exception as e:
    print('GPU setup note:', e)

# Audio & feature params
SR = 22050
DURATION = 3.0
TARGET_SAMPLES = int(SR * DURATION)
N_FFT = 1024
HOP = 512
N_MELS = 128

# Training params (GTZAN-aligned defaults with Tabla tuning)
BATCH_SIZE = int(os.environ.get('TAB_BATCH', '48'))
EPOCHS = int(os.environ.get('TAB_EPOCHS', '70'))
PATIENCE = int(os.environ.get('TAB_PATIENCE', '10'))
LR = float(os.environ.get('TAB_LR', '5e-4'))
LABEL_SMOOTHING = float(os.environ.get('TAB_LABEL_SMOOTHING', '0.02'))
SPEC_AUGMENT = os.environ.get('TAB_SPEC_AUGMENT', '1') == '1'
SPEC_FREQ_MASK = int(os.environ.get('TAB_SPEC_FREQ_MASK', '4'))
SPEC_TIME_MASK = int(os.environ.get('TAB_SPEC_TIME_MASK', '8'))
SPEC_NUM_MASKS = int(os.environ.get('TAB_SPEC_NUM_MASKS', '1'))
TAB_INIT_FROM_GTZAN = os.environ.get('TAB_INIT_FROM_GTZAN', '1') == '1'
TAB_FORCE_RETRAIN = os.environ.get('TAB_FORCE_RETRAIN', '0') == '1'
TAB_RESUME_FROM_CKPT = os.environ.get('TAB_RESUME_FROM_CKPT', '0') == '1'
SKIP_TRAIN_IF_CKPT = os.environ.get('SKIP_TRAIN_IF_CKPT', '0') == '1'

print('Config OK', {
    'BATCH': BATCH_SIZE, 'EPOCHS': EPOCHS, 'PATIENCE': PATIENCE, 'LR': LR,
    'SMOOTH': LABEL_SMOOTHING, 'SPEC_AUG': SPEC_AUGMENT,
    'SPEC_MASKS': (SPEC_FREQ_MASK, SPEC_TIME_MASK, SPEC_NUM_MASKS),
    'INIT_FROM_GTZAN': TAB_INIT_FROM_GTZAN,
    'FORCE_RETRAIN': TAB_FORCE_RETRAIN, 'RESUME': TAB_RESUME_FROM_CKPT, 'SKIP_
})

```

No GPUs visible or CPU forced; running on CPU.

Config OK {'BATCH': 48, 'EPOCHS': 70, 'PATIENCE': 10, 'LR': 0.0005, 'SMOOTH': 0.02, 'SPEC_AUG': True, 'SPEC_MASKS': (8, 16, 1), 'INIT_FROM_GTZAN': True, 'FORCE_RETRAIN': False, 'RESUME': False, 'SKIP_IF_CHKPT': False}

```
In [67]: # 2) Paths & Load CSV splits
def find_project_root(start: Path, markers=('requirements.txt', 'README.md')):
    cur = start.resolve()
    for p in [cur] + list(cur.parents):
        if all((p / m).exists() for m in markers):
            return p
    return start

PROJECT_ROOT = find_project_root(Path.cwd())
DATA_DIR = PROJECT_ROOT / 'data'
PROC_DIR = DATA_DIR / 'processed_tabla'
MODELS_DIR = PROJECT_ROOT / 'models'
REPORTS_DIR = PROJECT_ROOT / 'reports'
MODELS_DIR.mkdir(parents=True, exist_ok=True)
REPORTS_DIR.mkdir(parents=True, exist_ok=True)

train_csv = PROC_DIR / 'train_split.csv'
val_csv = PROC_DIR / 'val_split.csv'
test_csv = PROC_DIR / 'test_split.csv'
le_json = PROC_DIR / 'label_encoder.json'
assert train_csv.exists() and val_csv.exists() and test_csv.exists(), 'Split

train_df = pd.read_csv(train_csv)
val_df = pd.read_csv(val_csv)
test_df = pd.read_csv(test_csv)
with open(le_json) as f:
    classes = json.load(f)['classes']
num_classes = len(classes)
label2idx = {c: i for i, c in enumerate(classes)}
print('Classes:', classes)
print('Split sizes -> train:', len(train_df), 'val:', len(val_df), 'test:',

# Sanitize label strings: strip whitespace
for df in (train_df, val_df, test_df):
    df['label'] = df['label'].astype(str).str.strip()

# Basic sanity
assert 'audio_path' in train_df.columns and 'label' in train_df.columns, 'CS

# Validate: labels present in encoder
enc_set = set(classes)
train_set = set(train_df['label'].unique())
val_set = set(val_df['label'].unique())
test_set = set(test_df['label'].unique())
missing_in_encoder = sorted((train_set | val_set | test_set) - enc_set)
if missing_in_encoder:
    print('Warning: labels in splits not present in encoder:', missing_in_er
missing_in_splits = sorted(enc_set - (train_set | val_set | test_set))
print('Labels missing in any split (ok if expected):', missing_in_splits)

# Class distribution diagnostics
```

```

print('\nClass counts (train):')
print(train_df['label'].value_counts())
print('\nClass counts (val):')
print(val_df['label'].value_counts())
print('\nClass counts (test):')
print(test_df['label'].value_counts())

# Validate audio paths exist
for name, df in [('train', train_df), ('val', val_df), ('test', test_df)]:
    missing_paths = (~df['audio_path'].astype(str).apply(lambda p: Path(p).exists()))
    if missing_paths.any():
        print(f'Warning: {missing_paths} missing audio files in {name} split')

print('Paths OK')

```

Classes: ['Addhatrital', 'Bhajani', 'Dadra', 'Deepchandi', 'Ektal', 'Jhaptal', 'Rupak', 'Trital']

Split sizes -> train: 448 val: 56 test: 57

Labels missing in any split (ok if expected): []

Class counts (train):

label	count
Trital	79
Addhatrital	62
Dadra	58
Bhajani	57
Deepchandi	48
Ektal	48
Jhaptal	48
Rupak	48

Name: count, dtype: int64

Class counts (val):

label	count
Trital	10
Addhatrital	8
Bhajani	7
Dadra	7
Jhaptal	6
Rupak	6
Deepchandi	6
Ektal	6

Name: count, dtype: int64

Class counts (test):

label	count
Trital	10
Bhajani	8
Addhatrital	8
Dadra	7
Rupak	6
Deepchandi	6
Jhaptal	6
Ektal	6

Name: count, dtype: int64

Paths OK

```
In [68]: def spec_augment_tf(S, freq_mask_param=SPEC_FREQ_MASK, time_mask_param=SPEC_
# S: (n_mels, time)
shape = tf.shape(S, out_type=tf.int32)
n_mels = tf.gather(shape, 0)
t = tf.gather(shape, 1)

def mask_axis(x, max_width, axis):
    width = tf.random.uniform([], minval=0, maxval=max_width + 1, dtype=
    if axis == 0:
        # Frequency masking
        start_max = tf.maximum(n_mels - width, 1)
        start = tf.random.uniform([], 0, start_max, dtype=tf.int32, seed=
        after = tf.maximum(n_mels - start - width, 0)

        mask = tf.concat([
            tf.ones(tf.stack([start, t]), dtype=x.dtype),
            tf.zeros(tf.stack([width, t]), dtype=x.dtype),
            tf.ones(tf.stack([after, t]), dtype=x.dtype),
        ], axis=0)
    else:
        # Time masking
        start_max = tf.maximum(t - width, 1)
        start = tf.random.uniform([], 0, start_max, dtype=tf.int32, seed=
        after = tf.maximum(t - start - width, 0)

        mask = tf.concat([
            tf.ones(tf.stack([n_mels, start]), dtype=x.dtype),
            tf.zeros(tf.stack([n_mels, width]), dtype=x.dtype),
            tf.ones(tf.stack([n_mels, after]), dtype=x.dtype),
        ], axis=1)

    return x * mask

S = mask_axis(S, freq_mask_param, axis=0)
S = mask_axis(S, time_mask_param, axis=1)
return S
```

```
In [77]: # 4) Load precomputed arrays (processed_tabla) and build fast tf.data pipeli
from pathlib import Path
import numpy as np, tensorflow as tf

# Paths
PROJECT_ROOT = (Path.cwd()).resolve().parents[1]
PROCESSED_TABLA = PROJECT_ROOT / 'data' / 'processed_tabla'

# Arrays (already scaled in 00_Setup notebook)
X_train = np.load(PROCESSED_TABLA/'X_train.npy', mmap_mode='r')
y_train = np.load(PROCESSED_TABLA/'y_train.npy', mmap_mode='r')
X_val = np.load(PROCESSED_TABLA/'X_val.npy', mmap_mode='r')
y_val = np.load(PROCESSED_TABLA/'y_val.npy', mmap_mode='r')
X_test = np.load(PROCESSED_TABLA/'X_test.npy', mmap_mode='r')
y_test = np.load(PROCESSED_TABLA/'y_test.npy', mmap_mode='r')

# Ensure channel dim (N, M, T, 1)
if X_train.ndim == 3: X_train = X_train[..., None]
```

```

if X_val.ndim == 3: X_val = X_val[..., None]
if X_test.ndim == 3: X_test = X_test[..., None]

print('Loaded arrays:', X_train.shape, X_val.shape, X_test.shape)

# Vectorized SpecAugment (batch) like INDIAN

def batch_spec_augment(mels, freq_mask_param=SPEC_FREQ_MASK, time_mask_param=
    B = tf.shape(mels)[0]
    M = tf.shape(mels)[1]
    T = tf.shape(mels)[2]
    x = mels
    for _ in range(num_masks):
        if freq_mask_param > 0:
            f = tf.random.uniform([B, 1, 1, 1], 0, freq_mask_param + 1, dtype=tf.float32)
            f = tf.minimum(f, M)
            f0_max = tf.maximum(M - f, 1)
            f0 = tf.cast(tf.floor(tf.random.uniform([B, 1, 1, 1]) * tf.cast(f0_max, tf.float32)), tf.int32)
            freq_idx = tf.reshape(tf.range(M, dtype=tf.int32), [1, M, 1, 1])
            freq_mask = (freq_idx >= f0) & (freq_idx < (f0 + f))
            freq_mask = tf.broadcast_to(freq_mask, [B, M, T, 1])
            x = tf.where(freq_mask, tf.zeros([], dtype=x.dtype), x)
        if time_mask_param > 0:
            t = tf.random.uniform([B, 1, 1, 1], 0, time_mask_param + 1, dtype=tf.float32)
            t = tf.minimum(t, T)
            t0_max = tf.maximum(T - t, 1)
            t0 = tf.cast(tf.floor(tf.random.uniform([B, 1, 1, 1]) * tf.cast(t0_max, tf.float32)), tf.int32)
            time_idx = tf.reshape(tf.range(T, dtype=tf.int32), [1, 1, T, 1])
            time_mask = (time_idx >= t0) & (time_idx < (t0 + t))
            time_mask = tf.broadcast_to(time_mask, [B, M, T, 1])
            x = tf.where(time_mask, tf.zeros([], dtype=x.dtype), x)
    return x

# Build datasets from arrays (fast)
AUTOTUNE = tf.data.AUTOTUNE

def ds_with_optional_aug(X, y_int, batch_size, training: bool):
    ds = tf.data.Dataset.from_tensor_slices((X, y_int))
    if training:
        ds = ds.shuffle(min(10000, X.shape[0]), seed=RANDOM_STATE, reshuffle_each_iteration=True)
        # Do not drop remainder for val/test; only drop on training for stable E
        ds = ds.batch(batch_size, drop_remainder=training)
        # One-hot inside the pipeline to avoid expanding memory upfront
        def _one_hot(mel, y):
            return mel, tf.one_hot(tf.cast(y, tf.int32), depth=num_classes)
        ds = ds.map(_one_hot, num_parallel_calls=AUTOTUNE)
        if training and SPEC_AUGMENT:
            def _aug(mel, y):
                mel = batch_spec_augment(mel, SPEC_FREQ_MASK, SPEC_TIME_MASK, num_masks)
                return mel, y
            ds = ds.map(_aug, num_parallel_calls=AUTOTUNE, deterministic=False)
        return ds.prefetch(AUTOTUNE)

train_ds = ds_with_optional_aug(X_train, y_train, BATCH_SIZE, training=True)
val_ds = ds_with_optional_aug(X_val, y_val, BATCH_SIZE, training=False)

```

```

test_ds = ds_with_optional_aug(X_test, y_test, BATCH_SIZE, training=False)

# Input shape inferred from arrays
N_MELS_FIXED, T_FRAMES = int(X_train.shape[1]), int(X_train.shape[2])
print('Input shape will be:', (N_MELS_FIXED, T_FRAMES, 1))
print('Datasets ready (array-based)')

```

Loaded arrays: (448, 128, 130, 1) (56, 128, 130, 1) (57, 128, 130, 1)
Input shape will be: (128, 130, 1)
Datasets ready (array-based)

```

In [78]: # 5) Define UNet model (same architecture as GTZAN/INDIAN)
from keras import layers, models

def _unet_encoder_block(input_tensor, filters, pool=True, name_prefix=""):
    x = layers.Conv2D(filters, 3, padding='same', use_bias=False, name=f'{name_prefix}_conv')(input_tensor)
    x = layers.BatchNormalization(name=f'{name_prefix}_bn1')(x)
    x = layers.PReLU(shared_axes=[1, 2], name=f'{name_prefix}_prelu1')(x)
    x = layers.Conv2D(filters, 3, padding='same', use_bias=False, name=f'{name_prefix}_conv2')(x)
    x = layers.BatchNormalization(name=f'{name_prefix}_bn2')(x)
    x = layers.PReLU(shared_axes=[1, 2], name=f'{name_prefix}_prelu2')(x)
    skip_connection = x
    if pool:
        pool_output = layers.MaxPooling2D(2, name=f'{name_prefix}_pool')(x)
        return pool_output, skip_connection
    else:
        return x, skip_connection

def build_unet_audio_classifier(input_shape, num_classes):
    inputs = layers.Input(shape=input_shape, name='input_mel')
    # Encoder path
    p1, s1 = _unet_encoder_block(inputs, 32, name_prefix="enc1")
    p2, s2 = _unet_encoder_block(p1, 64, name_prefix="enc2")
    p3, s3 = _unet_encoder_block(p2, 128, name_prefix="enc3")
    # Bottleneck (pool=False)
    bottleneck, _ = _unet_encoder_block(p3, 256, pool=False, name_prefix="bottleneck")
    # Classification head
    x = layers.GlobalAveragePooling2D(name="gap")(bottleneck)
    x = layers.Dropout(0.5, name='dropout')(x)
    outputs = layers.Dense(num_classes, activation='softmax', dtype='float32')(x)
    return models.Model(inputs=inputs, outputs=outputs, name='UNet_Audio_Classifier')

def try_load_backbone_from_gtzan(model, models_dir: Path, classes: int) -> Model:
    """Optionally initialize encoder weights from a WITH_AUG GTZAN UNet checkpoint.

    We only load layers that exist with matching shapes, skipping the final
    """
    if not TAB_INIT_FROM_GTZAN:
        return model
    candidates = [
        'UNet_Audio_Classifier_best_WITH_AUG.keras',
        'UNet_Audio_Classifier_best_NO_AUG.keras',
    ]

```

```

for name in candidates:
    ckpt = (models_dir / name)
    if ckpt.exists():
        try:
            # Load full model to extract layer weights
            src = keras.models.load_model(ckpt.as_posix(), compile=False)
            loaded = 0
            for layer in model.layers:
                if layer.name == 'logits':
                    continue # skip final classification head
                try:
                    src_layer = src.get_layer(layer.name)
                    if src_layer is not None and len(layer.get_weights()) > 0:
                        # Ensure shapes match per weight tensor
                        tgt_w = layer.get_weights()
                        src_w = src_layer.get_weights()
                        if all(ti.shape == si.shape for ti, si in zip(tgt_w, src_w)):
                            layer.set_weights(src_w)
                            loaded += 1
                except Exception:
                    pass
            print(f'Loaded {loaded} compatible layer(s) from {ckpt.name}')
            return
        except Exception as e:
            print('Backbone init failed for', ckpt.name, '→', e)
    print('No compatible GTZAN checkpoint found for backbone init.')

# Use observed constants for input shape contract
inp_shape = (N_MELS_FIXED, T_FRAMES, 1)
model = build_unet_audio_classifier(inp_shape, num_classes)
try_load_backbone_from_gtzan(model, MODELS_DIR, num_classes)
model.summary()
print('Model ready (UNet_Audio_Classifier)')

```

Loaded 29 compatible layer(s) from UNet_Audio_Classifier_best_WITH_AUG.keras
Model: "UNet_Audio_Classifier"

Layer (type)	Output Shape	Par
input_mel (InputLayer)	(None, 128, 130, 1)	
enc1_conv1 (Conv2D)	(None, 128, 130, 32)	
enc1_bn1 (BatchNormalization)	(None, 128, 130, 32)	
enc1_prelu1 (PReLU)	(None, 128, 130, 32)	
enc1_conv2 (Conv2D)	(None, 128, 130, 32)	9
enc1_bn2 (BatchNormalization)	(None, 128, 130, 32)	
enc1_prelu2 (PReLU)	(None, 128, 130, 32)	
enc1_pool (MaxPooling2D)	(None, 64, 65, 32)	
enc2_conv1 (Conv2D)	(None, 64, 65, 64)	18
enc2_bn1 (BatchNormalization)	(None, 64, 65, 64)	
enc2_prelu1 (PReLU)	(None, 64, 65, 64)	
enc2_conv2 (Conv2D)	(None, 64, 65, 64)	36
enc2_bn2 (BatchNormalization)	(None, 64, 65, 64)	
enc2_prelu2 (PReLU)	(None, 64, 65, 64)	
enc2_pool (MaxPooling2D)	(None, 32, 32, 64)	
enc3_conv1 (Conv2D)	(None, 32, 32, 128)	73
enc3_bn1 (BatchNormalization)	(None, 32, 32, 128)	
enc3_prelu1 (PReLU)	(None, 32, 32, 128)	
enc3_conv2 (Conv2D)	(None, 32, 32, 128)	147
enc3_bn2 (BatchNormalization)	(None, 32, 32, 128)	
enc3_prelu2 (PReLU)	(None, 32, 32, 128)	
enc3_pool (MaxPooling2D)	(None, 16, 16, 128)	
bneck_conv1 (Conv2D)	(None, 16, 16, 256)	294
bneck_bn1 (BatchNormalization)	(None, 16, 16, 256)	1
bneck_prelu1 (PReLU)	(None, 16, 16, 256)	
bneck_conv2 (Conv2D)	(None, 16, 16, 256)	589
bneck_bn2 (BatchNormalization)	(None, 16, 16, 256)	1
bneck_prelu2 (PReLU)	(None, 16, 16, 256)	
gap (GlobalAveragePooling2D)	(None, 256)	

dropout (Dropout)	(None , 256)	
logits (Dense)	(None , 8)	2

Total params: 1,177,576 (4.49 MB)

Trainable params: 1,175,656 (4.48 MB)

Non-trainable params: 1,920 (7.50 KB)

Model ready (UNet_Audio_Classifier)

```
In [79]: # 6) Compile, class weights, and callbacks
opt = keras.optimizers.Adam(learning_rate=LR)
# Metrics: accuracy + top-3 like GTZAN
top3 = keras.metrics.TopKCategoricalAccuracy(k=3, name='top3_acc')
loss = keras.losses.CategoricalCrossentropy(label_smoothing=LABEL_SMOOTHING)
model.compile(optimizer=opt, loss=loss, metrics=['accuracy', top3])

# Class weights
train_labels = train_df['label'].astype(str).values
cw = compute_class_weight(class_weight='balanced', classes=np.array(classes))
class_weight = {i: float(w) for i, w in enumerate(cw)}
print('Class weights:', class_weight)

best_model_path = MODELS_DIR / 'UNet_Audio_Classifier_best_TABLA.keras'
csv_log_path = REPORTS_DIR / 'training_summary_TABLA.csv'
cb = [
    keras.callbacks.EarlyStopping(monitor='val_accuracy', patience=PATIENCE,
    keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patier
    keras.callbacks.ModelCheckpoint(best_model_path.as_posix(), monitor='val
]
print('Compile & callbacks ready')
```

Class weights: {0: 0.9032258064516129, 1: 0.9824561403508771, 2: 0.9655172413793104, 3: 1.1666666666666667, 4: 1.1666666666666667, 5: 1.1666666666666666, 6: 1.1666666666666667, 7: 0.7088607594936709}

Compile & callbacks ready

```
In [80]: # 7) Train or skip/resume based on checkpoint toggles
history = None
if TAB_FORCE_RETRAIN:
    print('FORCE_RETRAIN=1 → training from scratch, ignoring checkpoint')
    history = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS, cla
elif TAB_RESUME_FROM_CKPT and best_model_path.exists():
    print('RESUME_FROM_CKPT=1 → continuing training from checkpoint')
    try:
        model.load_weights(best_model_path.as_posix())
    except Exception as e:
        print('Failed to load checkpoint for resume:', e)
        history = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS, cla
elif SKIP_TRAIN_IF_CKPT and best_model_path.exists():
    print('Training skipped due to existing checkpoint.')
else:
    print('No checkpoint scenario → training from scratch')
    history = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS, cla
```

```
# Persist compact training summary
if history is not None:
    import numpy as np
    best_val = float(np.max(history.history.get('val_accuracy', [0])))
    epochs_run = int(len(history.history.get('val_accuracy', [])))
else:
    best_val = None
    epochs_run = 0

pd.DataFrame([
    {
        'Model': 'UNet_Audio_Classifier', 'Dataset': 'TABLA',
        'Best_Val_Accuracy': best_val,
        'Test_Accuracy': None, # filled after test eval
        'Epochs_Run': epochs_run
    }
]).to_csv(csv_log_path, index=False)
print('Saved:', csv_log_path)
```

No checkpoint scenario → training from scratch

Epoch 1/70

9/9  **13s** 1s/step - accuracy: 0.2269 - loss: 1.9857 - top 3_acc: 0.5440 - val_accuracy: 0.1429 - val_loss: 2.6647 - val_top3_acc: 0.5000 - learning_rate: 5.0000e-04

Epoch 2/70

9/9  **13s** 1s/step - accuracy: 0.5787 - loss: 1.3290 - top 3_acc: 0.8819 - val_accuracy: 0.2500 - val_loss: 4.0171 - val_top3_acc: 0.4464 - learning_rate: 5.0000e-04

Epoch 3/70

9/9  **13s** 1s/step - accuracy: 0.6875 - loss: 0.9807 - top 3_acc: 0.9514 - val_accuracy: 0.2143 - val_loss: 6.0458 - val_top3_acc: 0.5179 - learning_rate: 5.0000e-04

Epoch 4/70

9/9  **13s** 1s/step - accuracy: 0.8519 - loss: 0.6718 - top 3_acc: 0.9907 - val_accuracy: 0.3214 - val_loss: 3.6425 - val_top3_acc: 0.5179 - learning_rate: 5.0000e-04

Epoch 5/70

9/9  **13s** 1s/step - accuracy: 0.9190 - loss: 0.4655 - top 3_acc: 0.9977 - val_accuracy: 0.3571 - val_loss: 2.3888 - val_top3_acc: 0.6964 - learning_rate: 5.0000e-04

Epoch 6/70

9/9  **14s** 2s/step - accuracy: 0.9745 - loss: 0.3309 - top 3_acc: 1.0000 - val_accuracy: 0.4286 - val_loss: 1.9646 - val_top3_acc: 0.7321 - learning_rate: 5.0000e-04

Epoch 7/70

9/9  **14s** 2s/step - accuracy: 0.9838 - loss: 0.2707 - top 3_acc: 1.0000 - val_accuracy: 0.4821 - val_loss: 1.9998 - val_top3_acc: 0.7321 - learning_rate: 5.0000e-04

Epoch 8/70

9/9  **14s** 2s/step - accuracy: 0.9931 - loss: 0.2241 - top 3_acc: 1.0000 - val_accuracy: 0.4821 - val_loss: 2.0843 - val_top3_acc: 0.7321 - learning_rate: 5.0000e-04

Epoch 9/70

9/9  **13s** 1s/step - accuracy: 0.9931 - loss: 0.2147 - top 3_acc: 1.0000 - val_accuracy: 0.4464 - val_loss: 2.0154 - val_top3_acc: 0.6607 - learning_rate: 5.0000e-04

Epoch 10/70

9/9  **14s** 2s/step - accuracy: 0.9954 - loss: 0.2004 - top 3_acc: 1.0000 - val_accuracy: 0.5000 - val_loss: 1.6678 - val_top3_acc: 0.7143 - learning_rate: 5.0000e-04

Epoch 11/70

9/9  **13s** 1s/step - accuracy: 0.9977 - loss: 0.1821 - top 3_acc: 1.0000 - val_accuracy: 0.5893 - val_loss: 1.1729 - val_top3_acc: 0.8393 - learning_rate: 5.0000e-04

Epoch 12/70

9/9  **13s** 1s/step - accuracy: 1.0000 - loss: 0.1779 - top 3_acc: 1.0000 - val_accuracy: 0.6250 - val_loss: 0.9150 - val_top3_acc: 0.9286 - learning_rate: 5.0000e-04

Epoch 13/70

9/9  **13s** 1s/step - accuracy: 0.9977 - loss: 0.1688 - top 3_acc: 1.0000 - val_accuracy: 0.6429 - val_loss: 0.9060 - val_top3_acc: 0.9286 - learning_rate: 5.0000e-04

Epoch 14/70

9/9  **13s** 1s/step - accuracy: 1.0000 - loss: 0.1701 - top 3_acc: 1.0000 - val_accuracy: 0.6250 - val_loss: 0.9616 - val_top3_acc: 0.96

43 - learning_rate: 5.0000e-04
Epoch 15/70
9/9  13s 1s/step - accuracy: 1.0000 - loss: 0.1610 - top
3_acc: 1.0000 - val_accuracy: 0.7143 - val_loss: 0.7194 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 16/70
9/9  13s 1s/step - accuracy: 1.0000 - loss: 0.1597 - top
3_acc: 1.0000 - val_accuracy: 0.9286 - val_loss: 0.4538 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 17/70
9/9  13s 1s/step - accuracy: 1.0000 - loss: 0.1581 - top
3_acc: 1.0000 - val_accuracy: 0.9464 - val_loss: 0.3956 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 18/70
9/9  13s 1s/step - accuracy: 1.0000 - loss: 0.1560 - top
3_acc: 1.0000 - val_accuracy: 0.9464 - val_loss: 0.3439 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 19/70
9/9  13s 1s/step - accuracy: 0.9977 - loss: 0.1568 - top
3_acc: 1.0000 - val_accuracy: 0.9464 - val_loss: 0.3470 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 20/70
9/9  13s 1s/step - accuracy: 1.0000 - loss: 0.1537 - top
3_acc: 1.0000 - val_accuracy: 0.8571 - val_loss: 0.4582 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 21/70
9/9  13s 2s/step - accuracy: 1.0000 - loss: 0.1576 - top
3_acc: 1.0000 - val_accuracy: 0.9464 - val_loss: 0.3364 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 22/70
9/9  14s 2s/step - accuracy: 1.0000 - loss: 0.1540 - top
3_acc: 1.0000 - val_accuracy: 1.0000 - val_loss: 0.1883 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 23/70
9/9  13s 1s/step - accuracy: 1.0000 - loss: 0.1543 - top
3_acc: 1.0000 - val_accuracy: 1.0000 - val_loss: 0.1777 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 24/70
9/9  14s 2s/step - accuracy: 1.0000 - loss: 0.1539 - top
3_acc: 1.0000 - val_accuracy: 1.0000 - val_loss: 0.1692 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 25/70
9/9  13s 1s/step - accuracy: 1.0000 - loss: 0.1527 - top
3_acc: 1.0000 - val_accuracy: 1.0000 - val_loss: 0.1954 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 26/70
9/9  14s 2s/step - accuracy: 1.0000 - loss: 0.1481 - top
3_acc: 1.0000 - val_accuracy: 1.0000 - val_loss: 0.1771 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 27/70
9/9  14s 2s/step - accuracy: 0.9977 - loss: 0.1472 - top
3_acc: 1.0000 - val_accuracy: 1.0000 - val_loss: 0.1626 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 28/70
9/9  13s 1s/step - accuracy: 0.9977 - loss: 0.1546 - top
3_acc: 1.0000 - val_accuracy: 1.0000 - val_loss: 0.1554 - val_top3_acc: 1.00

```

00 - learning_rate: 5.0000e-04
Epoch 29/70
9/9 ----- 13s 1s/step - accuracy: 1.0000 - loss: 0.1513 - top
3_acc: 1.0000 - val_accuracy: 1.0000 - val_loss: 0.1667 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 30/70
9/9 ----- 14s 2s/step - accuracy: 1.0000 - loss: 0.1489 - top
3_acc: 1.0000 - val_accuracy: 1.0000 - val_loss: 0.1639 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 31/70
9/9 ----- 14s 2s/step - accuracy: 1.0000 - loss: 0.1476 - top
3_acc: 1.0000 - val_accuracy: 1.0000 - val_loss: 0.1597 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Epoch 32/70
9/9 ----- 13s 1s/step - accuracy: 1.0000 - loss: 0.1430 - top
3_acc: 1.0000 - val_accuracy: 1.0000 - val_loss: 0.1664 - val_top3_acc: 1.00
00 - learning_rate: 5.0000e-04
Saved: /home/alepot55/Desktop/projects/naml_project/reports/training_summary
_TABLA.csv

```

In [81]: *# 8) Evaluate on val & test, save reports (and update summary)*

```

from sklearn.metrics import accuracy_score

# Ensure we evaluate best checkpoint
if best_model_path.exists():
    try:
        model.load_weights(best_model_path.as_posix())
    except Exception as e:
        print('Warning: could not reload best checkpoint:', e)

# VAL on full array to avoid dropping last batch
val_probs = model.predict(X_val, batch_size=BATCH_SIZE, verbose=0)
val_pred_idx = np.argmax(val_probs, axis=1)
val_true_idx = np.asarray(y_val, dtype=int)
val_report = classification_report(val_true_idx, val_pred_idx, target_names=
print('[VAL]')
print(val_report)
with open(REPORTS_DIR / 'classification_report_UNet_Audio_Classifier_TABLA_V
    f.write(val_report)
cm_val = confusion_matrix(val_true_idx, val_pred_idx, labels=list(range(len(
plt.figure(figsize=(8,6))
sns.heatmap(cm_val, annot=True, fmt='d', cmap='Blues', xticklabels=classes,
plt.title('Confusion Matrix - VAL')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.tight_layout()
plt.savefig(REPORTS_DIR / 'confusion_matrix_UNet_Audio_Classifier_TABLA_VAL.
plt.close()

# TEST on full array
y_pred_probs = model.predict(X_test, batch_size=BATCH_SIZE, verbose=0)
y_pred_idx = np.argmax(y_pred_probs, axis=1)
y_true_idx = np.asarray(y_test, dtype=int)

# Save classification report (TEST)

```

```

report = classification_report(y_true_idx, y_pred_idx, target_names=classes,
with open(REPORTS_DIR / 'classification_report_UNet_Audio_Classifier_TABLA_T
    f.write(report)
print('[TEST]')
print(report)

# Save confusion matrix (TEST)
cm = confusion_matrix(y_true_idx, y_pred_idx, labels=list(range(len(classes))
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=classes, ytic
plt.title('Confusion Matrix - TEST')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.tight_layout()
plt.savefig(REPORTS_DIR / 'confusion_matrix_UNet_Audio_Classifier_TABLA_TEST
plt.close()

# Update training summary with final test accuracy
final_test_acc = accuracy_score(y_true_idx, y_pred_idx)
try:
    df = pd.read_csv(REPORTS_DIR/'training_summary_TABLA.csv')
    df.loc[0, 'Test_Accuracy'] = float(final_test_acc)
    df.to_csv(REPORTS_DIR/'training_summary_TABLA.csv', index=False)
except Exception as e:
    pd.DataFrame([{'Model':'UNet_Audio_Classifier', 'Dataset':'TABLA', 'Bes
print('Updated summary with Test_Accuracy:', final_test_acc)
print('Reports saved to', REPORTS_DIR)

```

2025-08-25 16:37:19.707206: E tensorflow/core/framework/node_def_util.cc:680] NodeDef mentions attribute use_unbounded_threadpool which is not in the op definition: Op<name=MapDataset; signature=input_dataset:variant, other_arguments: -> handle:variant; attr=f:func; attr=Targuments:list(type),min=0; attr=output_types:list(type),min=1; attr=output_shapes:list(shape),min=1; attr=use_inter_op_parallelism:bool,default=true; attr=preserve_cardinality:bool,default=false; attr=force_synchronous:bool,default=false; attr=metadata:string,default=""> This may be expected if your graph generating binary is newer than this binary. Unknown attributes will be ignored. NodeDef: {{node ParallelMapDatasetV2/_14}}

[VAL]	precision	recall	f1-score	support
Addhatrital	1.0000	1.0000	1.0000	8
Bhajani	1.0000	1.0000	1.0000	7
Dadra	1.0000	1.0000	1.0000	7
Deepchandi	1.0000	1.0000	1.0000	6
Ektal	1.0000	1.0000	1.0000	6
Jhaptal	1.0000	1.0000	1.0000	6
Rupak	1.0000	1.0000	1.0000	6
Trital	1.0000	1.0000	1.0000	10
accuracy			1.0000	56
macro avg	1.0000	1.0000	1.0000	56
weighted avg	1.0000	1.0000	1.0000	56

[TEST]	precision	recall	f1-score	support
Addhatrital	1.0000	1.0000	1.0000	8
Bhajani	1.0000	1.0000	1.0000	8
Dadra	1.0000	1.0000	1.0000	7
Deepchandi	1.0000	0.8333	0.9091	6
Ektal	0.7500	1.0000	0.8571	6
Jhaptal	1.0000	0.8333	0.9091	6
Rupak	1.0000	1.0000	1.0000	6
Trital	1.0000	1.0000	1.0000	10
accuracy			0.9649	57
macro avg	0.9688	0.9583	0.9594	57
weighted avg	0.9737	0.9649	0.9658	57

Updated summary with Test_Accuracy: 0.9649122807017544
 Reports saved to /home/alepot55/Desktop/projects/naml_project/reports